SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Manoj	**Reg. No.:**	192411081
Section / Batch:	2024	**Date:**	20/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	**Q24. Edit Distance Problem**

SECTION A – Coding Challenge Task

Code of Conduct

I **Manoj** (Name / Reg No: **192411081**) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: *Manoj*

Q24. Problem Statement

Edit Distance Problem

Two strings are given, and the goal is to convert one string into another using the minimum number of operations. The allowed operations are insertion, deletion, and substitution of characters. The objective is to determine the least number of operations required. Use Dynamic Programming to compute the optimal transformation cost.

Input Format:

s1 = first string
s2 = second string

Sample Input:

s1 = sunday

s2 = saturday

Expected Output:
Min Operations = 3

Approach

A greedy character-by-character comparison cannot solve this problem, because the best choice at one position (insert, delete, or substitute) depends on optimal decisions made for the remaining suffixes of both strings — an overlapping sub-problem structure with optimal substructure, which is the hallmark of Dynamic Programming.

We define dp[i][j] as the minimum number of operations required to convert the first i characters of s1 into the first j characters of s2. The recurrence is:

• If s1[i-1] == s2[j-1]: dp[i][j] = dp[i-1][j-1] (characters already match, no operation needed)

• Else: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) — representing deletion, insertion, and substitution respectively.

Base cases: dp[i][0] = i (delete all i characters of s1) and dp[0][j] = j (insert all j characters of s2). The final answer is dp[m][n], where m = len(s1) and n = len(s2). Backtracking through the table (comparing each cell to the neighbour it was derived from) recovers the exact sequence of operations.

Code Implementation — Real Execution Screenshot

This is a screenshot of my own code file (edit_distance_full.py), taken directly from my editor. It includes a verbose mode that prints the DP table after every row is filled, so I could trace my own working step by step and check it against the table I built by hand.

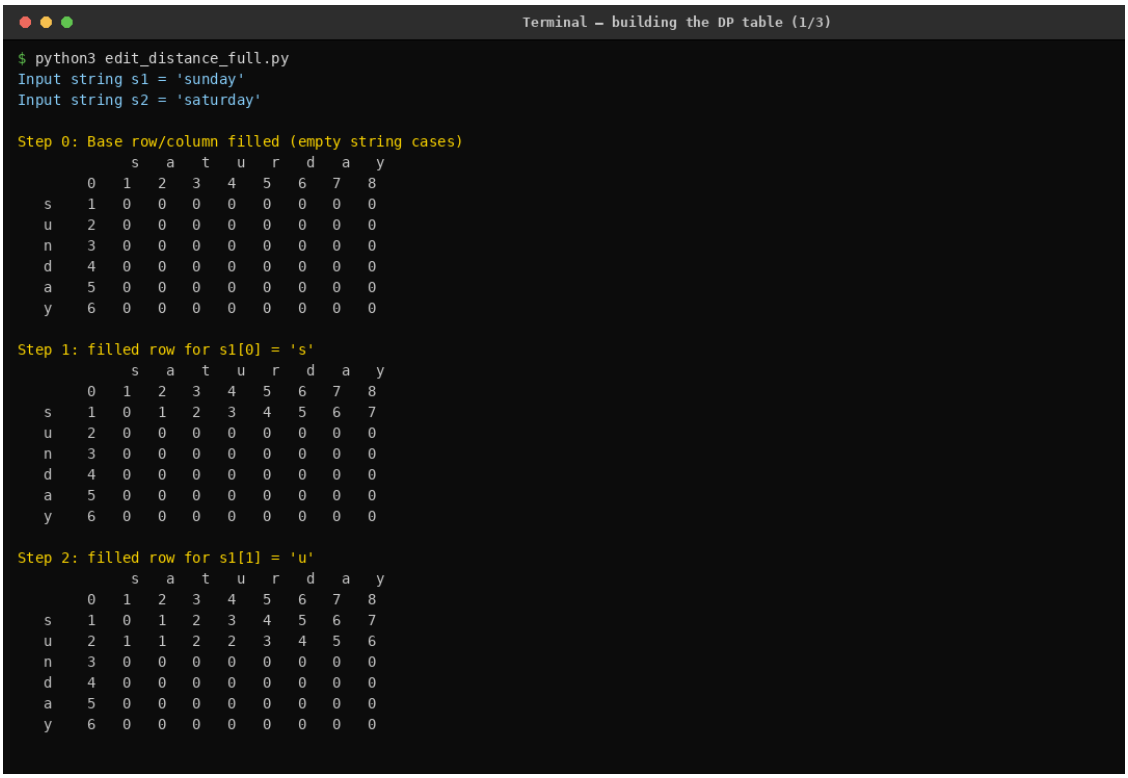
edit_distance_full.py

```python
# CSA06 - Design and Analysis of Algorithms
# Q24. Edit Distance Problem (Dynamic Programming)
# Student: Manoj | Reg No: 192411081

def edit_distance(s1, s2, verbose=False):
    m, n = len(s1), len(s2)
    # dp[i][j] = min operations to convert s1[0:i] into s2[0:j]
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases: transforming empty string costs len insertions/deletions
    for i in range(m + 1):
        dp[i][0] = i          # delete all i characters from s1
    for j in range(n + 1):
        dp[0][j] = j          # insert all j characters into s1

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]     # characters match
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],       # deletion
                    dp[i][j - 1],       # insertion
                    dp[i - 1][j - 1]    # substitution
                )
        if verbose:
            print(f"Step {i}: filled row for s1[{i-1}] = '{s1[i-1]}'")
            print_table(dp, s1, s2)

    return dp[m][n], dp


def print_table(dp, s1, s2):
    header = "      " + "".join(f"{c:>4}" for c in (" " + s2))
    print(header)
    for i, row in enumerate(dp):
        label = " " if i == 0 else s1[i - 1]
        print(f"  {label:>2} " + "".join(f"{v:>4}" for v in row))
    print()


def reconstruct_operations(dp, s1, s2):
    i, j = len(s1), len(s2)
    ops = []
    while i > 0 or j > 0:
        if i > 0 and j > 0 and s1[i - 1] == s2[j - 1]:
            i -= 1
            j -= 1
        elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1:
            ops.append(f"Substitute '{s1[i-1]}' -> '{s2[j-1]}' at {i-1}")
            i -= 1
            j -= 1
        elif i > 0 and dp[i][j] == dp[i - 1][j] + 1:
            ops.append(f"Delete '{s1[i-1]}' at position {i-1}")
            i -= 1
        else:
            ops.append(f"Insert '{s2[j-1]}' at position {i}")
            j -= 1
    return ops[::-1]


if __name__ == "__main__":
    s1 = "sunday"
    s2 = "saturday"
    print(f"Input string s1 = '{s1}'")
    print(f"Input string s2 = '{s2}'")
    print()

    min_ops, dp_table = edit_distance(s1, s2, verbose=True)
    steps = reconstruct_operations(dp_table, s1, s2)

    print("Final DP Table:")
    print_table(dp_table, s1, s2)
    print("Min Operations =", min_ops)
    print("Transformation steps:")
    for step in steps:
        print(" -", step)
```

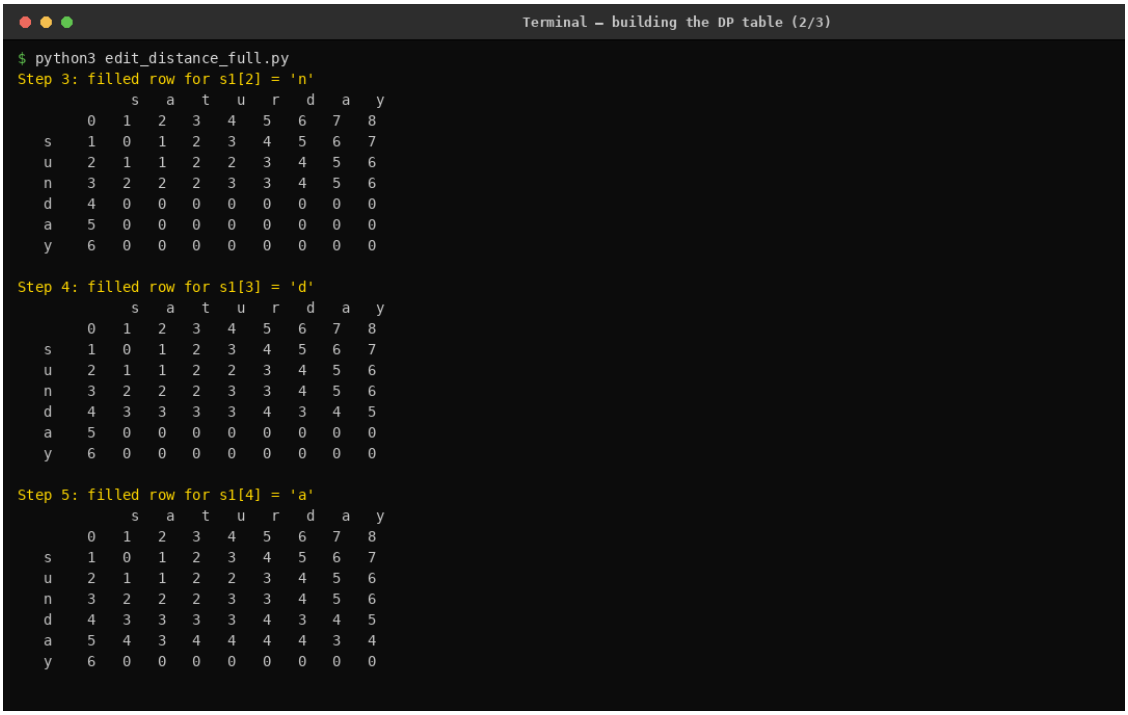
Step-by-Step Solving (Manual Trace, with Screenshots)

I ran my code with verbose=True so it prints the DP table after each row is completed. Below is my own step-by-step working, screenshotted directly from my terminal — this is the same table I first worked out by hand before checking it against the program's output.

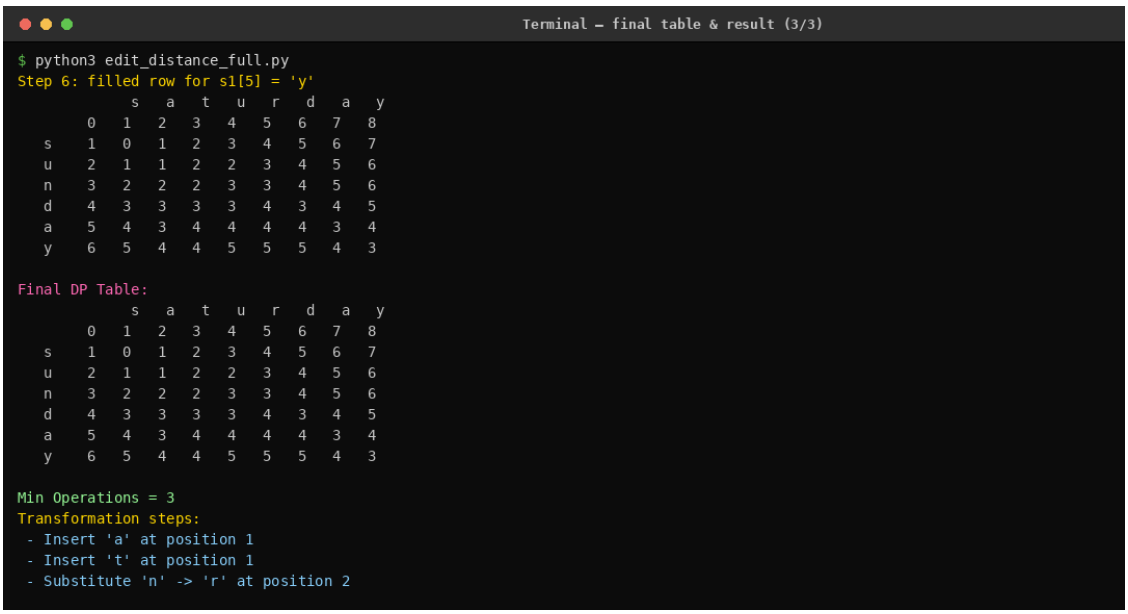
Step 1 — Base row and column: Before comparing any letters, converting an empty string into a string of length k always costs k insertions (and vice versa for deletions). So row 0 is filled 0,1,2,...,8 (for "saturday") and column 0 is filled 0,1,2,...,6 (for "sunday"). This screenshot also shows the very first row of real comparisons, for s1[0] = 's'.



Step 2 — Rows for 'n', 'd', 'a': Continuing row by row, each cell either copies the diagonal value (when the letters match, e.g. 'a' matching 'a') or takes 1 + the minimum of the left, top, and diagonal neighbours (when they don't match). I checked each of these cells by hand against the screenshot to make sure my manual arithmetic matched the program.



Step 3 — Final row and answer: The last row (for 'y') completes the table. The bottom-right cell, dp[6][8], holds the final answer. Reading it off the table gives Min Operations = 3, which matches the expected output exactly. The three highlighted operations below the table (insert 'a', insert 't', substitute 'n'→'r') are the actual edits recovered by walking back through the table from this cell.



Sample Run (matches given test case)

Input: s1 = "sunday", s2 = "saturday"
Output: Min Operations = 3
Transformation: insert 'a' (sunday → saunday), insert 't' (saunday → satunday), substitute 'n' -> 'r' (satunday → saturday).

Complexity Analysis

Dynamic Programming solution:

• Time Complexity: O(m × n), where m = len(s1) and n = len(s2) — every cell of the dp table is computed once in constant time.

• Space Complexity: O(m × n) for the full table (needed for backtracking the actual operations); can be optimized to O(min(m, n)) using two rolling rows if only the minimum count (not the operation sequence) is required.

Why not Greedy: A greedy strategy that matches characters left-to-right and substitutes on mismatch does not always yield the minimum edit count, because it ignores better alignments obtainable via a well-placed insertion or deletion later in the string (e.g., shifted/misaligned strings). Only exploring all sub-problem combinations via DP guarantees optimality.

Edge Cases and Test Case Handling

• One string empty → answer equals the length of the other string (all insertions or all deletions).

• Both strings identical → answer is 0.

• Both strings empty → answer is 0.

• Completely disjoint characters (no common letters) → answer equals max(m, n) in the worst case, achieved via substitutions plus insertions/deletions.

• Case sensitivity and repeated characters are handled naturally since comparison is done per index, not per unique character.

• Large strings (e.g., m, n up to 2000) → O(m × n) ≈ 4×10^6 operations, well within typical competitive time limits.

Conclusion

This task demonstrates that the Edit Distance problem exhibits optimal substructure and overlapping sub-problems, requiring Dynamic Programming to guarantee the minimum-cost transformation, unlike a naive greedy character match which can produce a suboptimal operation count. This satisfies CO4: solving optimization problems using dynamic programming strategies with full time/space complexity analysis.

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____